Cloud Platforms (AWS)

Session 4: Containers & Docker

MIBA24 · Master of Science in Business Analytics

ESADE Business School | February 25, 2026

Oriol Rius Canals | joseporiol.rius@esade.edu

Welcome & Session Overview

From databases to deployable applications

Course Progress

S1	S2	S3	S4	S5	S6	S7	S8	S9
AWS Intro	SQL Basics	Advanced SQL	Containers	SageMaker I	SageMaker II	APIs	Project	Exam

Today: Session 4 Containerization

Today's Learning Objectives

Understand Why

Grasp why containers are essential for modern data and analytics workflows

Build Images

Create Docker images that package applications with their dependencies

Use Registries

Push and pull container images from GitHub Container Registry (GHCR)

Deploy Containers

Run containers in production using Docker Compose on cloud infrastructure

Focus: Understanding the business value of containers for analytics, not becoming DevOps experts

The Problem Containers Solve

Why packaging matters for analytics

"It Works on My Machine"

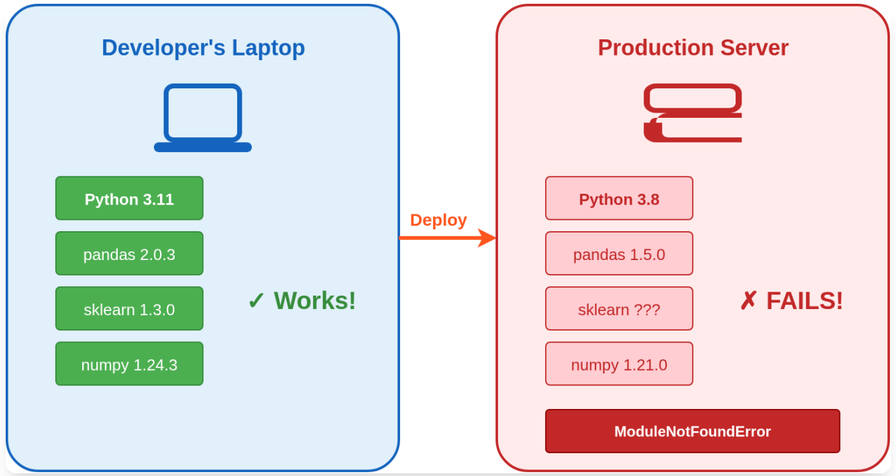
The Classic Problem

You develop a Python analytics script on your laptop:

```python
import pandas as pd
import sklearn
import numpy as np

# Your brilliant analysis...
model = train_model(data)
```

You send it to your colleague or deploy to a server...

> ⚠ **Error:**
> ModuleNotFoundError: No module named 'sklearn'



The most expensive phrase in software development

Dependency Hell in Analytics

Real-World Scenario

Your ML pipeline requires specific versions:

```
# Your development environment
Python 3.11
pandas==2.0.3
scikit-learn==1.3.0
numpy==1.24.3
tensorflow==2.13.0
```

```
# Production server has
Python 3.8
pandas==1.5.0
scikit-learn==1.2.0
numpy==1.21.0
tensorflow==2.10.0
```

⚠ **Result:** Subtle bugs, different model outputs, or complete failures. Your analysis isn't reproducible!

Dev Laptop

macOS, Python 3.11

Test Server

Ubuntu, Python 3.9

Production

Amazon Linux, Python 3.8

Traditional vs Container Deployment

How containers change the deployment game

Traditional Deployment

Manual Steps

- Install Python on server
- Install system dependencies
- Create virtual environment
- Install pip packages
- Configure environment variables
- Hope versions match
- Pray it works

Hours of setup per environment

Container Deployment

Single Command

- Build image once
- Push to registry
- Pull and run anywhere

```
docker run myapp:v1.0
```

Seconds to deploy

Key insight: Containers package your code AND all its dependencies together, guaranteeing identical behavior everywhere.

The Business Case for Containers

Why Should Analytics Teams Care?

- **Reproducibility** - Same results every time
- **Collaboration** - Share complete environments
- **Speed** - Deploy in seconds, not hours
- **Rollback** - Instantly revert to previous versions
- **Scale** - Run 1 or 100 instances identically

Real Impact

Netflix: Deploys thousands of containers per day

Data Science Teams: 10x faster experiment iteration

$ Cost Savings: Reduced environment setup from days to minutes

The Container Promise

Build Once, Run Anywhere

Your Laptop | Test Server | AWS Cloud | Colleague's PC

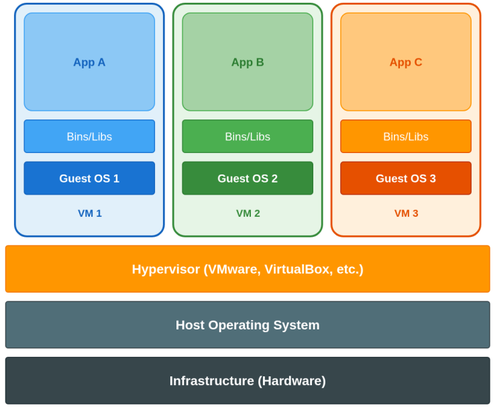
Same Image | Same Behavior | Same Results

Container Fundamentals

Understanding the building blocks

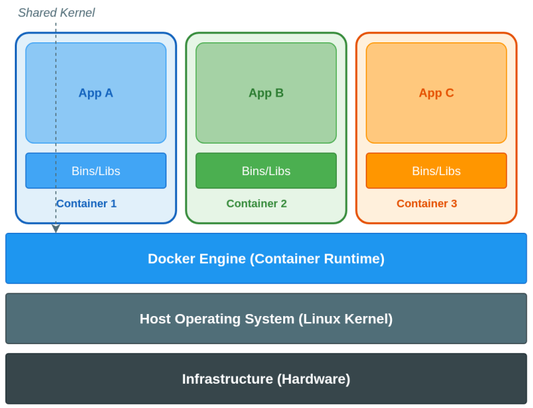
Containers vs Virtual Machines

Virtual Machine



- Full OS per application
- Heavy (gigabytes)
- Boot time: minutes
- Strong isolation

Container



- Shared OS kernel
- Lightweight (megabytes)
- Start time: seconds
- Process-level isolation

Analogy: VMs are like separate houses (each with full infrastructure). Containers are like apartments (shared building, private spaces).

Image vs Container

Docker Image

Blueprint / Recipe

- Read-only template
- Contains code + dependencies
- Can be shared and versioned
- Stored in registries

```
myapp:v1.0.0  (image tag)
├── Python 3.11
├── pandas, sklearn
├── Your application code
└── Configuration files
```

Docker Container

Running Instance

- Created from an image
- Has its own filesystem
- Can be started/stopped
- Multiple from same image

```
# One image, multiple containers
docker run myapp:v1.0.0  # Container 1
docker run myapp:v1.0.0  # Container 2
docker run myapp:v1.0.0  # Container 3
```

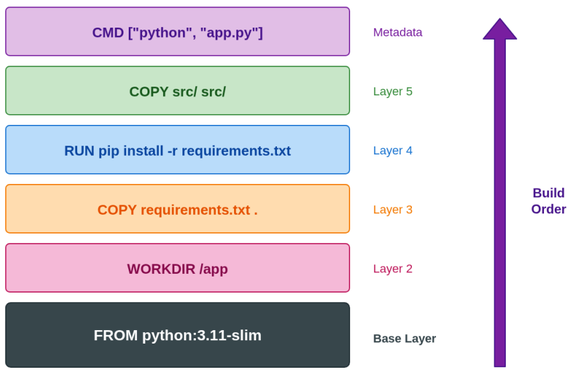
Think of it like: Image = Class definition, Container = Object instance

Container Layers

How Images Are Built

Images are built in **layers**, each representing a change:

```
FROM python:3.11-slim       # Base layer
WORKDIR /app                # New layer
COPY requirements.txt .     # New layer
RUN pip install -r ...      # New layer
COPY src/ src/              # New layer
CMD ["python", "app.py"]    # Metadata
```



Each instruction creates a new layer

Layers are cached for faster rebuilds

Layers are shared between images

Efficient Storage

Layers are shared between images

Fast Builds

Only changed layers rebuild

Quick Pulls

Download only missing layers

What's Inside a Container?

Container Anatomy

1. **Base Operating System**
   Minimal Linux (Alpine, Debian, Ubuntu)
2. **Runtime Environment**
   Python, Node.js, Java, etc.
3. **Dependencies**
   Libraries, packages, tools
4. **Application Code**
   Your scripts and programs
5. **Configuration**
   Environment variables, settings

Example: Analytics Container

```
analytics-pipeline:v1.0
│
├── /usr (Base OS: Debian slim)
│   └── /bin, /lib...
│
├── /usr/local/bin/python3.11
│   └── Python interpreter
│
├── /usr/local/lib/python3.11/
│   └── site-packages/
│       ├── pandas/
│       ├── numpy/
│       └── sklearn/
│
├── /app (Your code)
│   ├── pipeline.py
│   ├── models/
│   └── config.yaml
│
└── Environment Variables
    ├── DB_HOST=rds.amazonaws.com
    └── API_KEY=***
```

Key Container Concepts

Isolation

Each container runs in its own namespace, separated from others and the host system

Lightweight

Containers share the host OS kernel, making them much smaller than VMs

Portable

Run the same container on any system with Docker installed

For analytics: These properties mean you can develop locally and deploy to AWS with confidence that your code behaves identically.

Docker in Practice

Building and running containers

The Dockerfile

Your Container Recipe

A **Dockerfile** is a text file with instructions to build an image:

```
# Start from a base image
FROM python:3.13-slim

# Set working directory
WORKDIR /app

# Copy and install dependencies
COPY requirements.txt .
RUN pip install -r requirements.txt

# Copy application code
COPY src/ src/

# Define the startup command
CMD ["python", "src/main.py"]
```

Instruction Reference

Instruction	Purpose
`FROM`	Base image to start from
`WORKDIR`	Set working directory
`COPY`	Copy files into image
`RUN`	Execute commands (build time)
`ENV`	Set environment variables
`EXPOSE`	Document ports
`CMD`	Default command (runtime)

Best practice: Order instructions from least to most frequently changed for better layer caching.

Real-World Dockerfile

hello-world Project Dockerfile

```
FROM python:3.13-slim

LABEL org.opencontainers.image.source=https://github.com/oriol
LABEL org.opencontainers.image.description="Simple hello-world
LABEL org.opencontainers.image.licenses=MIT

# Install uv
COPY --from=ghcr.io/astral-sh/uv:latest /uv /usr/local/bin/uv
# Set working directory
WORKDIR /app
# Copy project files
COPY pyproject.toml README.md ./
COPY src/ src/
# Install the application
RUN uv pip install --system .
# Expose port
EXPOSE 49000
# Run the server
CMD ["hello-world", "--bind", "0.0.0.0", "--port", "49000"]
```

Key Points

1. **Base Image**
   Official Python slim image (smaller, secure)
2. **Labels**
   OCI metadata for registry (source, description, license)
3. **Install uv**
   Copy uv binary from official image (multi-stage pattern)
4. **Application Setup**
   Set workdir, copy project files, install with uv
5. **Runtime Config**
   Expose port and define startup command

Building Docker Images

The Build Command

```
docker build -t myapp:v1.0 .
```

Part	Meaning
`docker build`	Command to build an image
`-t myapp:v1.0`	Tag (name:version) for the image
`.`	Build context (current directory)

Build Output

```
[+] Building 45.2s (12/12) FINISHED
 => [1/6] FROM python:3.13-slim
 => [2/6] WORKDIR /app
 => [3/6] COPY pyproject.toml ...
 => [4/6] COPY src/ src/
 => [5/6] RUN uv pip install ...
 => exporting to image
 => naming to myapp:v1.0
```

Useful Build Options

```
# Build with no cache (clean rebuild)
docker build --no-cache -t myapp:v1.0 .

# Build for specific platform
docker build --platform linux/amd64 \
  -t myapp:v1.0 .

# Build from specific Dockerfile
docker build -f docker/Dockerfile \
  -t myapp:v1.0 .
```

Running Containers

The Run Command

```
docker run -p 8080:49000 myapp:v1.0
```

Common Options

Option	Purpose	Example
`-p`	Port mapping	`-p 8080:49000`
`-d`	Detached mode	`-d`
`-e`	Environment var	`-e DB_HOST=...`
`--name`	Container name	`--name myapp`
`-v`	Mount volume	`-v ./data:/app/data`
`--rm`	Remove on exit	`--rm`

Examples

```
# Run interactively
docker run -it myapp:v1.0 bash

# Run in background with port mapping
docker run -d -p 8080:49000 myapp:v1.0

# Run with environment variables
docker run -e API_KEY=secret \
  -e DEBUG=true myapp:v1.0

# Run with volume mount
docker run -v $(pwd)/data:/app/data \
  myapp:v1.0
```

Port mapping explained: `-p HOST:CONTAINER` maps port 8080 on your machine to port 49000 inside the container.

Essential Docker Commands

Image Management

```
# List local images
docker images

# Remove an image
docker rmi myapp:v1.0

# Pull from registry
docker pull python:3.13-slim

# Push to registry
docker push ghcr.io/user/myapp:v1.0

# View image history/layers
docker history myapp:v1.0
```

Container Management

```
# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# Stop a container
docker stop <container_id>

# Remove a container
docker rm <container_id>

# View container logs
docker logs <container_id>

# Execute command in running container
docker exec -it <container_id> bash
```

>_ **Pro tip:** Use `docker system prune` to clean up unused images, containers, and networks.

Docker Commands Cheatsheet

Build

```
docker build -t name:tag .
```

Create image from Dockerfile

Run

```
docker run -d -p 80:80 name
```

Start a container

Stop

```
docker stop container_id
```

Gracefully stop container

List

```
docker ps -a
```

Show all containers

Logs

```
docker logs -f container_id
```

Stream container output

Exec

```
docker exec -it id bash
```

Run command in container

Container Registries

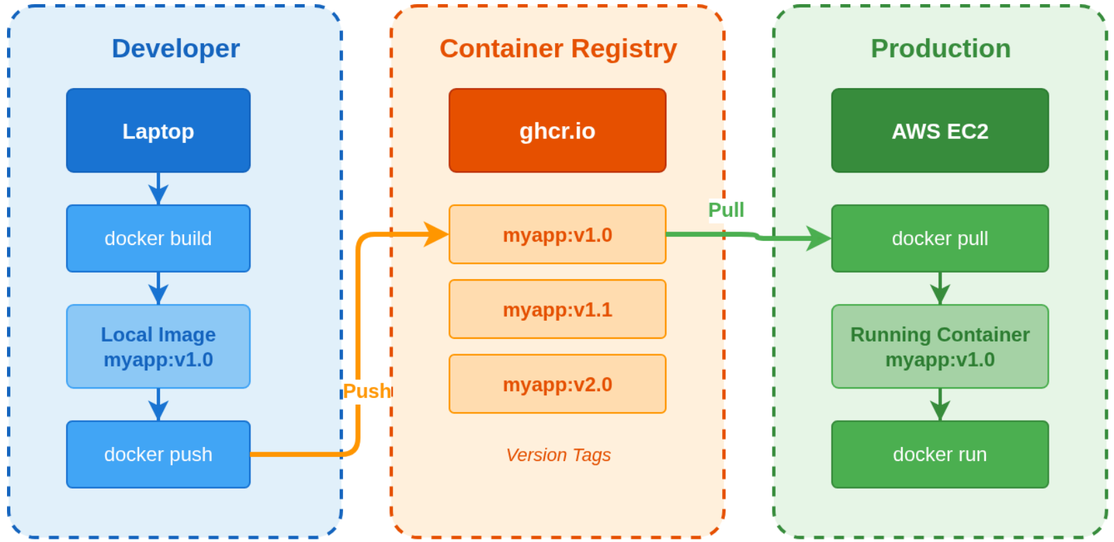
Storing and sharing container images

What is a Container Registry?

Think: App Store for Containers

A registry is a **storage and distribution system** for Docker images.

- **Push** - Upload your images
- **Pull** - Download images to run
- **Version** - Tag images with versions
- **Share** - Distribute to teams



Popular Container Registries

Docker Hub

Default public registry

hub.docker.com

Public | Free tier

GitHub Container Registry

Integrated with GitHub

ghcr.io

We use this!

Amazon ECR

AWS native registry

Elastic Container Registry

AWS integrated

Why GHCR for this course? It's free, integrates with GitHub Actions (our CI/CD), and doesn't require AWS credentials to pull public images.

Image Naming Convention

Full Image Reference

```
ghcr.io/oriolrius/hello-world:v4.0.0
└────┘ └──────┘ └──────────┘ └────┘
   |        |          |        |
Registry   Owner Repository    Tag
```

Components

Part	Example	Purpose
Registry	`ghcr.io`	Where image is hosted
Owner	`oriolrius`	User/organization
Repository	`hello-world`	Image name
Tag	`v4.0.0`	Version identifier

> ⚠ **Warning:**
> Avoid using `:latest` in production. It's mutable and can lead to unexpected updates. Always use specific version tags.

Tag Strategies

```
# Semantic versioning
myapp:v1.0.0
myapp:v1.0
myapp:v1
# Environment tags
myapp:production
myapp:staging
# Git-based
myapp:abc123f
myapp:main
# Special
myapp:latest  ← Default (avoid in prod!)
```

Push and Pull Workflow

Pushing to GHCR

```
# 1. Login to registry
echo $GITHUB_TOKEN | docker login ghcr.io -u USERNAME --passwo
# 2. Tag your image for the registry
docker tag myapp:v1.0 ghcr.io/username/myapp:v1.0
# 3. Push to registry
docker push ghcr.io/username/myapp:v1.0
```

Pulling from GHCR

```
# Pull specific version
docker pull ghcr.io/oriolrius/hello-world:v5.0.0
# Pull and run in one command
docker run ghcr.io/oriolrius/hello-world:v5.0.0
```

Public images: No login needed to pull public images from GHCR!

In GitHub Actions

```
- name: Login to GHCR
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Build and push
  uses: docker/build-push-action@v6
  with:
    push: true
    tags: ghcr.io/${{ github.repository }}:${{ github.ref_name
```

Best practice: Use `build-push-action` to build and push in one step with dynamic tags.

GHCR vs ECR Comparison

Both are excellent choices - we use GHCR for this course

GHCR (GitHub)

Advantages

- Free for public images
- Integrated with GitHub Actions
- Simple authentication (GITHUB_TOKEN)
- No AWS account needed to pull
- Familiar GitHub interface

Best For

- Open source projects
- GitHub-centric workflows
- Learning and experimentation

ECR (AWS)

Advantages

- Native AWS integration
- Private by default
- IAM-based access control
- Works with ECS, EKS, Lambda
- Vulnerability scanning

Best For

- Enterprise AWS deployments
- Private company images
- Production workloads on AWS

Course choice: GHCR simplifies our workflow since we're already using GitHub. In a production AWS environment, ECR would be the natural choice.

Container Orchestration

Managing containers at scale

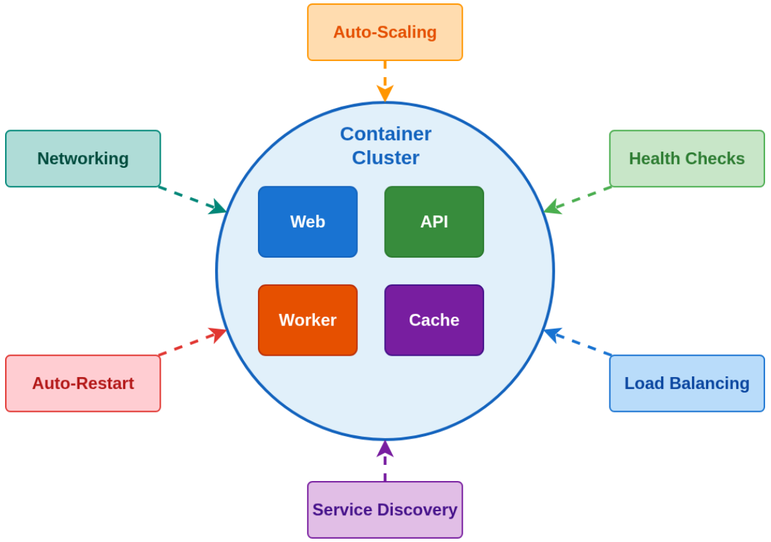
Why Orchestration?

Single Container is Easy

```
docker run myapp:v1.0
```

But Real Applications Need...

- Multiple services (web, database, cache)
- Automatic restarts on failure
- Scaling up/down based on load
- Service discovery and networking
- Rolling updates without downtime
- Health checks and self-healing



Docker Compose

Multi-Container Made Simple

Docker Compose lets you define and run multi-container applications with a single YAML file.

```yaml
# docker-compose.yml
services:
  web:
    image: myapp:v1.0
    ports:
      - "8080:80"
    environment:
      - DB_HOST=database
    depends_on:
      - database
  database:
    image: postgres:15
    environment:
      - POSTGRES_PASSWORD=secret
    volumes:
      - db-data:/var/lib/postgresql/data
volumes:
  db-data:
```

Key Features

1. **Service Definition**
   Define multiple containers as services
2. **Networking**
   Services can communicate by name
3. **Dependencies**
   Control startup order with depends_on
4. **Volumes**
   Persist data across restarts
5. **Single Command**
   `docker compose up -d` starts everything

Docker Compose Commands

Essential Commands

```
# Start all services (detached)
docker compose up -d

# Stop all services
docker compose down

# View running services
docker compose ps

# View logs
docker compose logs -f

# Restart a specific service
docker compose restart web

# Pull latest images
docker compose pull
```

Our hello-world Compose File

```yaml
services:
  hello-world:
    image: ghcr.io/oriolrius/hello-world:latest
    ports:
      - "49000:49000"
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f",
             "http://localhost:49000/"]
      interval: 30s
      timeout: 10s
      retries: 3
      start_period: 5s
```

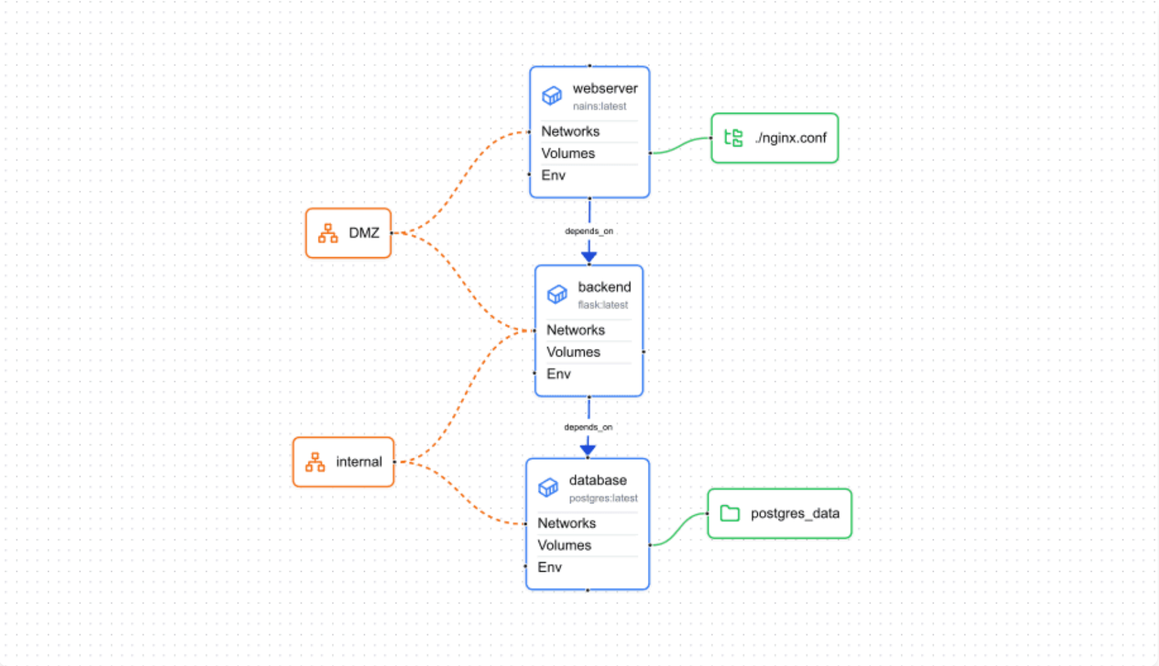
Health check: Docker monitors the container and restarts it if unhealthy.

Compose Craft

Visual Docker Compose builder — no YAML editing required.

- Import existing compose files
- Visualize service architecture
- Drag-and-drop editing
- Export valid YAML

> **Free tool:** composecraft.com — great for learning and documenting stacks.



Orchestration Options

Docker Compose

Complexity: Low

Scale: Single host

Best for: Development, small deployments

We use this

Amazon ECS

Complexity: Medium

Scale: Multi-host, managed

Best for: AWS-native production workloads

AWS Managed

Kubernetes

Complexity: High

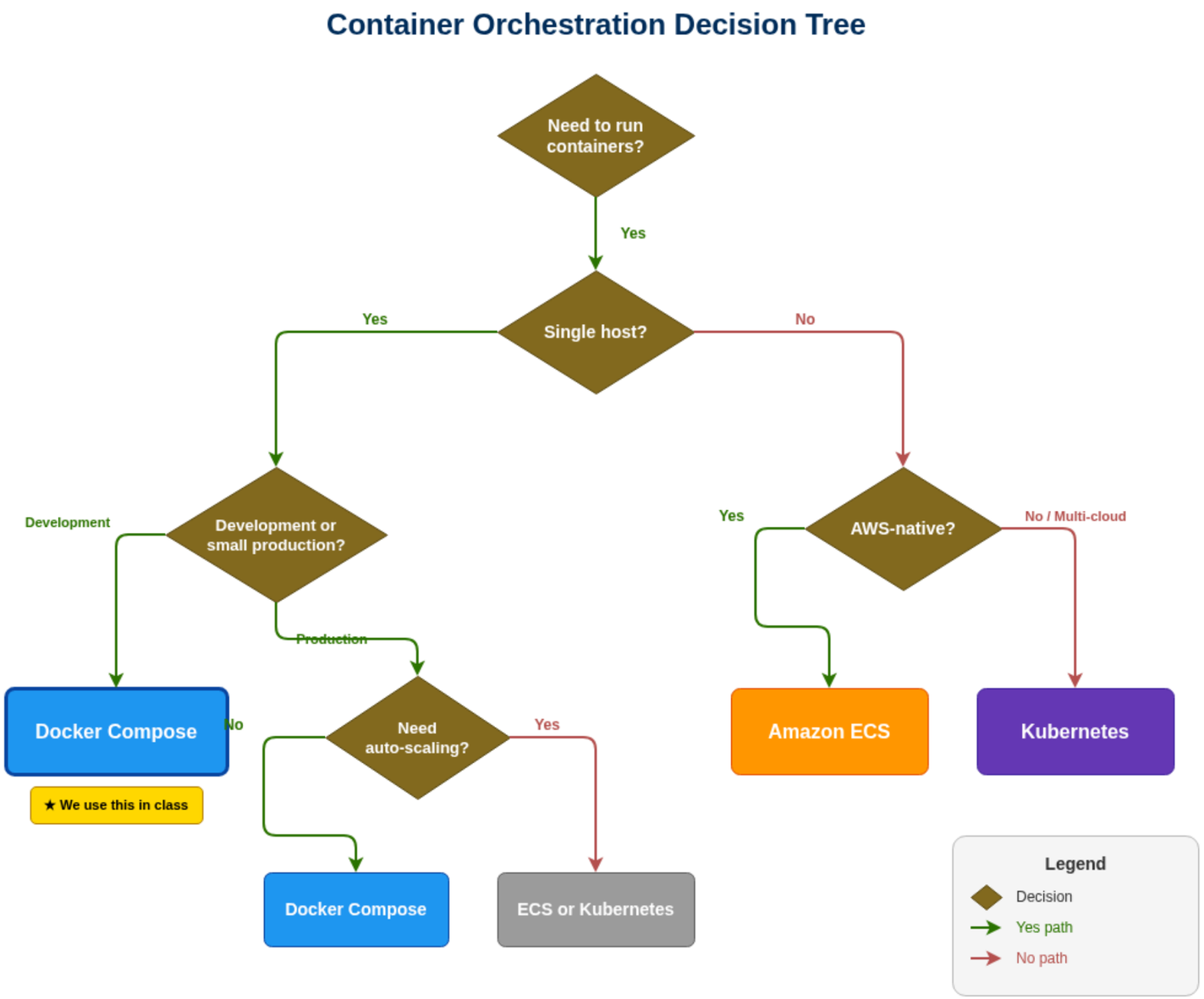
Scale: Planet-scale

Best for: Large, complex microservices

Industry standard

For analytics teams: Start with Docker Compose. Move to ECS when you need managed scaling. Kubernetes is typically overkill unless you're a platform team.

When to Use What?



Orchestration Concepts to Know

Health Checks

Periodic tests to verify container is working correctly

Auto-Restart

Automatically restart failed containers to maintain availability

Load Balancing

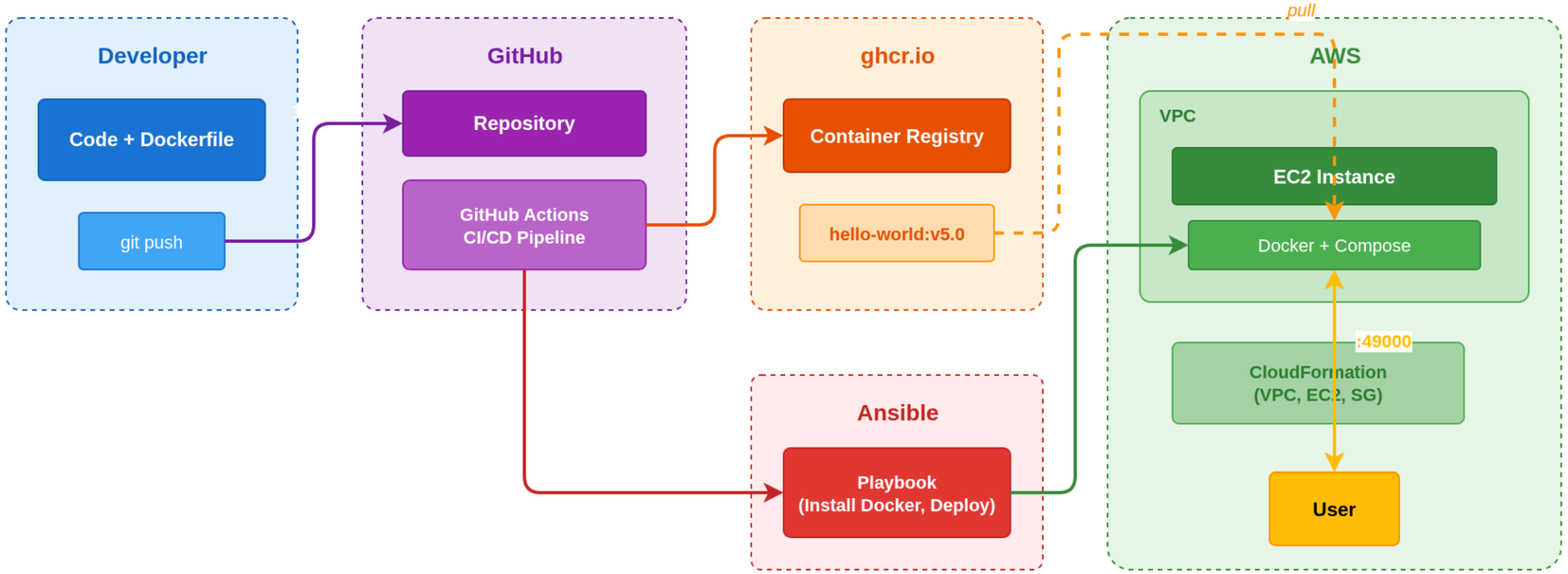
Distribute traffic across multiple container instances

Key takeaway: Orchestration ensures your containers stay running, healthy, and can handle varying loads without manual intervention.

Live Demo: hello-world Project

End-to-end containerized deployment

Project Architecture



1 Push code to GitHub

2 GitHub Actions builds image

3 Image pushed to GHCR

4 Ansible deploys to EC2

The Dockerfile Explained

```
1  FROM python:3.13-slim
2
3  LABEL org.opencontainers.image.source=\
4    https://github.com/oriolrius/hello-world
5  LABEL org.opencontainers.image.description=\
6    "Simple hello-world web server"
7  LABEL org.opencontainers.image.licenses=MIT
8
9  # Install uv (fast Python package manager)
10 COPY --from=ghcr.io/astral-sh/uv:latest \
11   /uv /usr/local/bin/uv
12
13 WORKDIR /app
14
15 # Copy project files
16 COPY pyproject.toml README.md ./
17 COPY src/ src/
18
19 # Install the application
20 RUN uv pip install --system .
21
22 EXPOSE 49000
23
24 CMD ["hello-world", "--bind", "0.0.0.0", \
25      "--port", "49000"]
```

Line-by-Line

Line 1: Start from Python 3.13 slim image

Lines 3-7: Add metadata labels for GHCR

Lines 9-10: Copy `uv` tool from another image (multi-stage copy)

Line 12: Set working directory inside container

Lines 14-16: Copy only what's needed

Line 18: Install app using uv

Line 20: Document the exposed port

Line 22-23: Define the startup command

> **Best practice:** Copy dependency files before code to leverage Docker's layer caching.

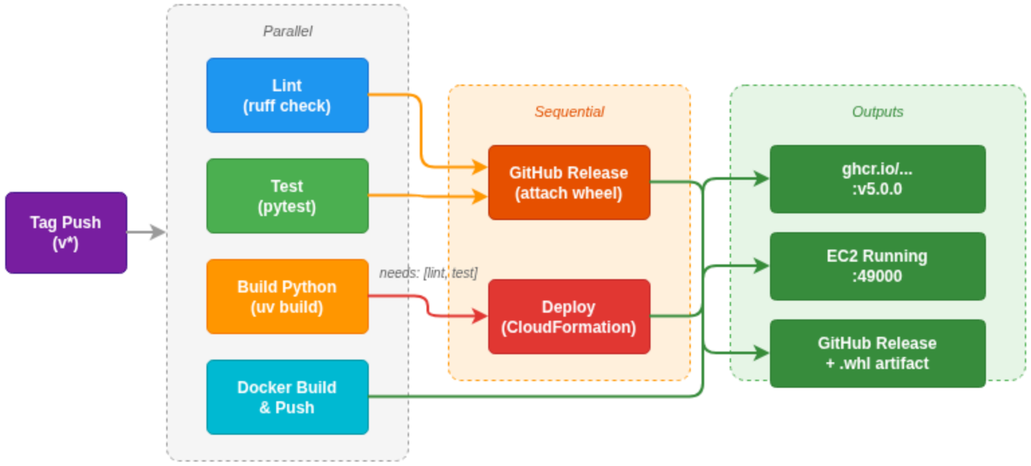
GitHub Actions CI/CD Pipeline

Pipeline Jobs (Conceptual)

```yaml
jobs:
  lint:         # Code quality checks (ruff)
  test:         # Run pytest

  build:        # Build Python package
    needs: [lint, test]

  release:      # Create GitHub release
    needs: [build]

  docker:       # Build & push to GHCR
    needs: [lint, test]

  deploy:       # Deploy via CloudFormation
    needs: [build, docker]
```

See full workflow: `.github/workflows/release.yml`

Pipeline Flow



Triggered on: Git tag push (v*) or manual dispatch

Docker Build & Push in Actions

```yaml
- name: Set up Docker Buildx
  uses: docker/setup-buildx-action@v3

- name: Log in to Container Registry
  uses: docker/login-action@v3
  with:
    registry: ghcr.io
    username: ${{ github.actor }}
    password: ${{ secrets.GITHUB_TOKEN }}

- name: Extract metadata for Docker
  id: meta
  uses: docker/metadata-action@v5
  with:
    images: ghcr.io/${{ github.repository }}
    tags: type=semver,pattern=v{{version}}

- name: Build and push Docker image
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: ${{ steps.meta.outputs.tags }}
```

Steps Explained

1. **Buildx** - Multi-platform build support
2. **Login** - Authenticate with GITHUB_TOKEN
3. **Metadata** - Extract tags from git (semver)
4. **Build & Push** - Single action does both

Key benefit: `metadata-action` auto-generates tags like `v1.0.0` from git tags.

Live Demo

Let's see it in action!

Review the code

Watch the pipeline

Access the service

View Repository

Containers for Business Analytics

Real-world applications

Analytics Use Cases for Containers

Why Analytics Teams Use Containers

- **Reproducible experiments** - Same environment across runs
- **Team collaboration** - Share complete environments
- **Dev to prod parity** - What works locally works in cloud
- **Dependency isolation** - Different projects, different requirements
- **Version control** - Roll back to any previous environment

Common Containerized Workloads

ML Model Serving

Package trained models with inference code

Data Pipelines

ETL jobs that run on schedule

Dashboard APIs

Backend services for BI tools

Jupyter Environments

Consistent notebook environments

Containerized ML Model

Example: Fraud Detection API

```
FROM python:3.11-slim

WORKDIR /app

# Install ML dependencies
COPY requirements.txt .
RUN pip install -r requirements.txt

# Copy trained model
COPY models/fraud_model.pkl models/

# Copy API code
COPY src/api.py .

EXPOSE 8000

CMD ["uvicorn", "api:app", "--host", "0.0.0.0"]
```

The API Code

```
from fastapi import FastAPI
import pickle
import pandas as pd

app = FastAPI()

# Load model once at startup
with open("models/fraud_model.pkl", "rb") as f:
    model = pickle.load(f)

@app.post("/predict")
def predict(transaction: dict):
    df = pd.DataFrame([transaction])
    prediction = model.predict(df)
    return {"fraud_probability": prediction[0]}
```

Result: Model + dependencies + API = one deployable container

Wrap-up

Key takeaways and next steps

What We Learned Today

The Why

Containers solve the "works on my machine" problem by packaging code with dependencies

The How

Dockerfiles define images, `docker build` creates them, `docker run` executes them

The Where

Container registries (GHCR, ECR) store and distribute images across environments

The Scale

Docker Compose orchestrates multi-container apps; ECS/K8s handle larger deployments

Core concept: Build once, run anywhere. Containers ensure your analytics code behaves identically across all environments.

Key Commands Reference

Building & Running

```
# Build an image
docker build -t myapp:v1.0 .

# Run a container
docker run -d -p 8080:80 myapp:v1.0

# List running containers
docker ps

# View logs
docker logs -f <container_id>

# Stop container
docker stop <container_id>
```

Registry & Compose

```
# Login to GHCR
echo $TOKEN | docker login ghcr.io -u USER --password-stdin

# Push to registry
docker push ghcr.io/user/app:v1.0

# Pull from registry
docker pull ghcr.io/user/app:v1.0

# Start compose services
docker compose up -d

# Stop compose services
docker compose down
```

Full reference: docs.docker.com/reference

Resources

Documentation

- Docker Official Docs
- GitHub Container Registry
- AWS ECR Documentation

Course Materials

- hello-world Repository
- eCampus Session 4 Materials

Learning Resources

- Docker 101 Tutorial
- Play with Docker - Free hands-on lab
- Docker in 100 Seconds - Quick overview

Tools

- Docker Desktop - Local development
- VS Code Docker Extension

Session 4 Assignment

This assignment is **significantly more complex** than previous sessions.

You will deploy a full CI/CD pipeline with GitHub Actions, AWS CloudFormation, and Docker containers.

View Complete Assignment Instructions

Fork & Deploy

Run the full CI/CD pipeline

Document

Explain each workflow step

Extend

Add whoami service

Submission: Upload the PDF to **eCampus Session 4 Assignment**

Questions?

Oriol Rius joseporiol.rius@esade.edu

Office Hours: Online at https://oriolrius.me/meeting

eCampus: Course materials and announcements

Thank You!

See you in Session 5: AWS SageMaker I